

Build Your Own IaC

Demystifying the Magic

We are going to put a web page on the internet using ~~AWS~~ Azure. Then we will build a tool to help us.

I have a web page. It needs to be on the internet.



Getting it there the hard way is how we learn what Terraform, Bicep and Pulumi actually do.

Everything we're going to make

resource group · byoiac-demo – the folder it all lives in

storage account · byoiacdemo2026 – a globally-unique name

container · files – a public bucket

hello.html – the page, with a real URL

Start with scripting

```
src/l-cli/deploy.sh

1  az group create --name byoiac-demo --location eastus --tags env=demo talk=kcdc
2
3  az storage account create --name byoiacdemo2026 --resource-group byoiac-demo \
4    --location eastus --sku Standard_LRS --kind StorageV2 --allow-blob-public-access true
5
6  az storage container create --name files --account-name byoiacdemo2026 --public-access blob
7
8  echo '<h1>Deployed by one dumb script</h1><p>KCDC 2026 - Build Your Own IaC</p>' > hello.html
9  az storage blob upload --account-name byoiacdemo2026 --container-name files \
10    --name hello.html --file hello.html --content-type text/html --overwrite
11
12 echo "https://byoiacdemo2026.blob.core.windows.net/files/hello.html"
```

This is what you'd write. Group, account, container, upload, print the URL.

Let's remove the tool dependency

```
src/2-api/deploy.py
38     f"/blobServices/default/containers/{CONTAINER}?api-version=2023-05-01",
39     {"properties": {"publicAccess": "Blob"}})
40
41 # 4. the file itself. blobs are the data plane: a different API, different auth,
42 #    so ask the management plane to sign us a SAS. still a PUT, though.
43 sas = call("POST", f"{ARM}/subscriptions/{SUB}/resourceGroups/{RG}"
44            f"/providers/Microsoft.Storage/storageAccounts/{ACCOUNT}"
45            f"/listServiceSas?api-version=2023-05-01",
46            {"canonicalizedResource": f"/blob/{ACCOUNT}/{CONTAINER}",
47            "signedResource": "c", "signedPermission": "rcwd", "signedProtocol": "https",
48            "signedExpiry": time.strftime("%Y-%m-%dT%H:%M:%SZ",
49                                          time.gmtime(time.time() + 3600))})["serviceSasToken"]
50
51 PAGE = "<h1>Deployed by one dumb script</h1><p>KCDC 2026 - Build Your Own IaC</p>"
52 blob_url = f"https://{ACCOUNT}.blob.core.windows.net/{CONTAINER}/{BLOB}"
53 print(f"PUT {blob_url}")
54 req = urllib.request.Request(f"{blob_url}?{sas}", method="PUT", data=PAGE.encode(),
55                              headers={"x-ms-version": "2021-08-06", "x-ms-blob-type": "BlockBlob",
56                                       "Content-Type": "text/html"})
57 with urllib.request.urlopen(req) as resp:
58     print(f" → {resp.status}")
59
60 print(f"\nyour page: {blob_url}")
```

Run it

zsh — byoiac

```
$ python src/2-api/deploy.py
```

```
PUT https://management.azure.com/subscriptions/<sub>/resourcegroups/byoiac-demo
```

```
→ 201
```

```
PUT https://management.azure.com/subscriptions/<sub>/resourceGroups/byoiac-demo/providers/Microsoft.Storage/storageAccounts/byoiacdemo2026
```

```
→ 202
```

```
sleeping 20s for the storage account ...
```

```
PUT ... /storageAccounts/byoiacdemo2026/blobServices/default/containers/files
```

```
→ 201
```

```
POST ... /storageAccounts/byoiacdemo2026/listServiceSas
```

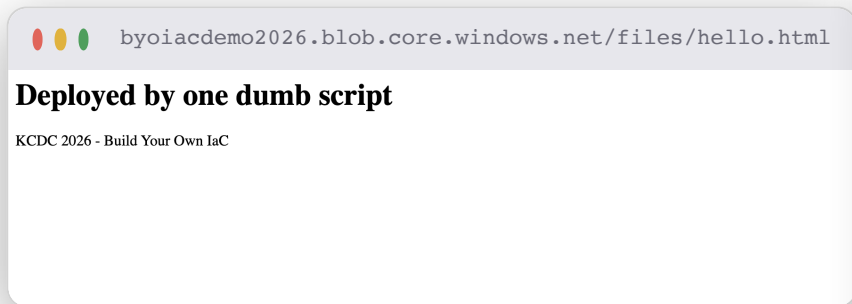
```
→ 200
```

```
PUT https://byoiacdemo2026.blob.core.windows.net/files/hello.html
```

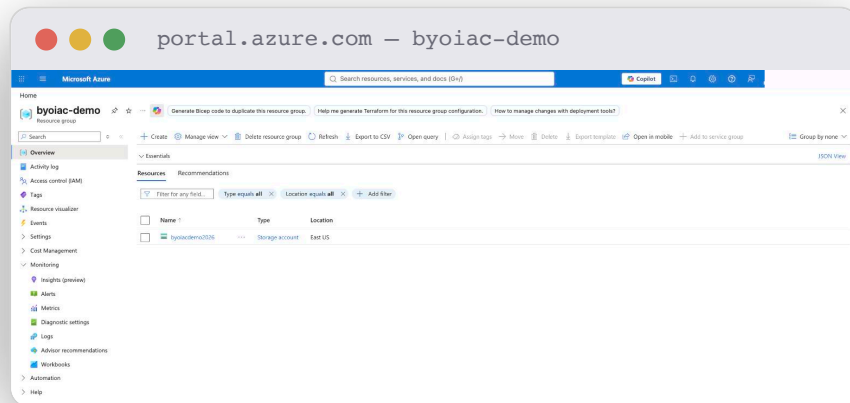
```
→ 201
```

```
your page: https://byoiacdemo2026.blob.core.windows.net/files/hello.html
```

Minute five: it's real



what the world sees



what Azure now holds

Failure one – run it again

```

zsh — byoiac

$ python src/2-api/deploy.py
PUT ... /resourcegroups/byoiac-demo
  → 200
PUT ... /storageAccounts/byoiacdemo2026
  → 200

sleeping 20s for the storage account ...

PUT ... /blobServices/default/containers/files
  → 200
POST ... /listServiceSas
  → 200
PUT https://byoiacdemo2026.blob.core.windows.net/files/hello.html
  → 201

```

It "worked": 200s instead of 201s, because PUT is an upsert. Nothing broke. It also re-sent everything and **sat there for twenty seconds** for a page that was already exactly right.

By the way, we got lucky

Azure: one verb

```
# create OR update – same call, same URL
call("PUT", f"{ARM}/ ... /storageAccounts/{ACCOUNT}",
     { ... })
```

ARM's PUT is an upsert. Rerunning was safe *by accident*.

Check what exists, then decide what to send. **Hold that thought. We're about to do it on purpose.**

AWS: pick the right verb

```
try:
    s3.create_bucket(Bucket=name) # exists? error
except ClientError:
    pass # ...probably fine?
lambda_.create_function( ... ) # exists? error
# vs update_function_configuration( ... )
```

Create-X or Update-X: to pick one, you must first ask **what's out there**.

Failure two – the page is no longer wanted

```
src/2-api/deploy.py

- # 3. the container – still ARM, just a deeper URL
- call("PUT", f"{ARM}/subscriptions/{SUB}/resourceGroups/{RG}"
-       f"/providers/Microsoft.Storage/storageAccounts/{ACCOUNT}"
-       f"/blobServices/default/containers/{CONTAINER}?api-version=2023-05-01",
-       {"properties": {"publicAccess": "Blob"}})
-
- print(f"PUT {blob_url}")
- req = urllib.request.Request(f"{blob_url}?{sas}", method="PUT", data=PAGE.encode(),
-                               headers={"x-ms-version": "2021-08-06", "x-ms-blob-type": "BlockBlob",
-                                         "Content-Type": "text/html"})
- with urllib.request.urlopen(req) as resp:
-     print(f" → {resp.status}")
```

Run it. The group and the account get PUT again. **The file is still sitting on the internet.**

Nothing happens, and *nothing can happen*. A script only ever says **make this exist**. No line you could add means *get rid of what I asked for yesterday*.

The obvious fix: a line for every delete

```
src/2-api/deploy.py  
  
+ # container + page removed 2026-08-31 - keep these lines forever  
+ call("DELETE", f"{ARM}/subscriptions/{SUB}/resourceGroups/{RG}"  
+       f"/providers/Microsoft.Storage/storageAccounts/{ACCOUNT}"  
+       f"/blobServices/default/containers/files?api-version=2023-05-01")
```

It works, once. But removing something now means **adding** a line.

Six months in, the script is creates on top, a graveyard of deletes below, re-deleting things that are long gone, every run. The script is a **log of actions**. Nobody can read what the system *is*.

How do you do nothing when nothing changed?
How do you delete?

We have a working solution. It's missing one idea.

What's already there? Then don't redo it.

What's no longer wanted? Then get rid of it.

Send **only the difference**.

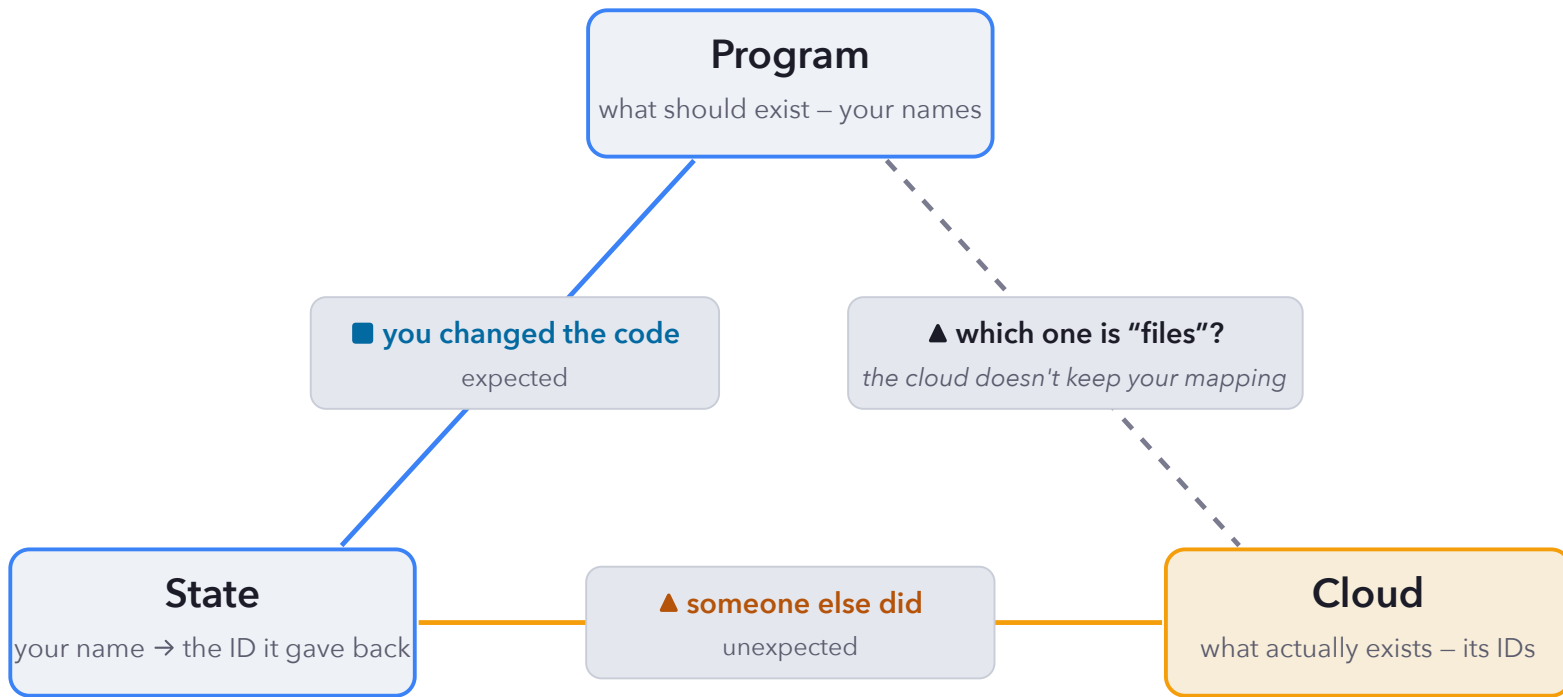
That's a **diff**. The rest of this talk is making it explicit.

The program becomes data

```
infra.yaml
1  resources:
2    rg:
3      type: Microsoft.Resources/resourceGroups
4      name: byoiac-demo
5      properties: {location: eastus, tags: {env: demo, talk: kcdc}}
6
7    storage:
8      type: Microsoft.Storage/storageAccounts
9      name: byoiacdemo2026
10     resourceGroup: byoiac-demo
11     properties: {location: eastus, sku: {name: Standard_LRS},
12                 kind: StorageV2, allowBlobPublicAccess: true}
13
14    files:
15      type: Microsoft.Storage/.../blobServices/containers
16      name: files
17      storageAccount: byoiacdemo2026
18      properties: {publicAccess: Blob}
19
20    hello:
21      type: blob
22      name: hello.html
23      container: files
24      properties: {contentType: text/html,
25                  content: "<h1>Deployed by ...</h1>"}
26
```

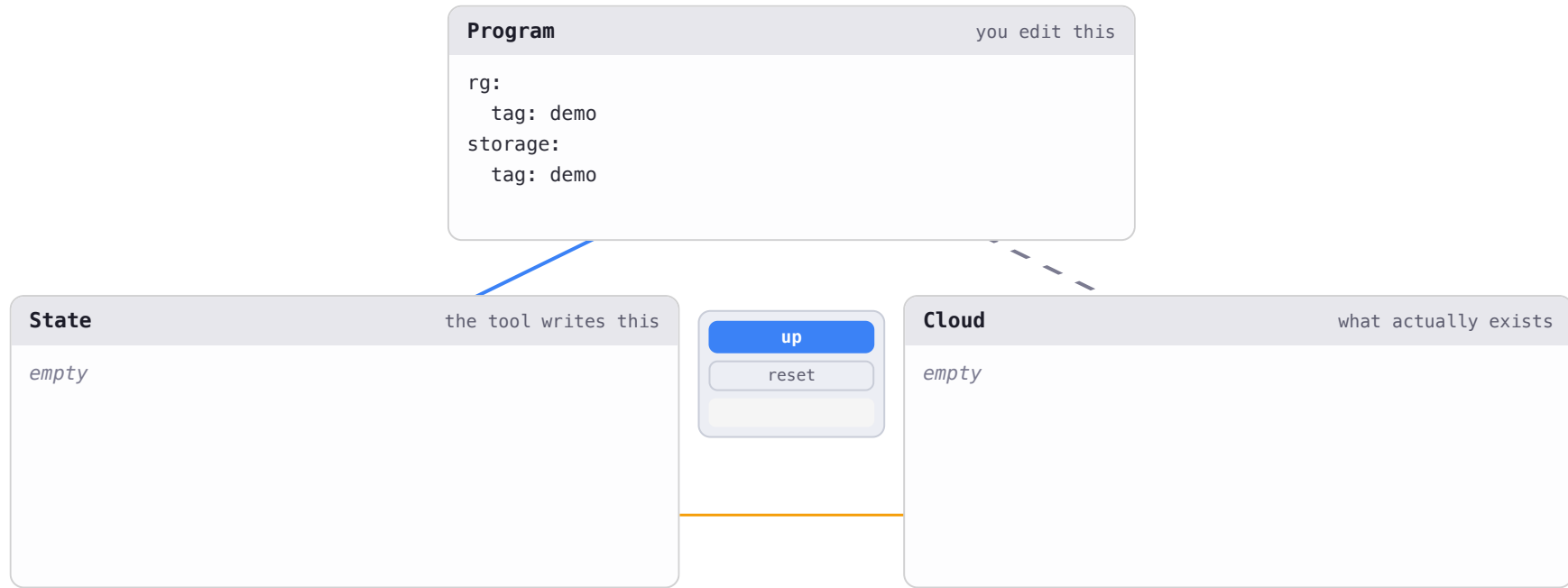
The script, with the verbs removed. No PUT, no sleep, no order, just what should exist.

The matching problem



The diff is only computable on top of this mapping.
State is memory of what you made, written down at create time.

The same picture, live



Step 3 – first, the memory

```
src/3-state/engine.py

1  STATE_FILE = "state.json"
2
3  # — pillar 1: state — what we THINK exists —————
4
5  def load_state():
6      return json.load(open(STATE_FILE)) if os.path.exists(STATE_FILE) else {}
7
8  def save_state(state):
9      json.dump(state, open(STATE_FILE, "w"), indent=2)
10
11 # ...and after every resource the loop touches, further down the file:
12
13     call("PUT", url_for(res), res["properties"])
14     state[key] = {"res": res, "saved": res["properties"]}
15     save_state(state)
```

The whole answer to "what's mine?" is a **JSON file the tool writes as it works.**

Step 3 – reconcile: compare, then act

```
src/3-state/engine.py

1  def diff(desired, state):
2      creates = [k for k in desired if k not in state]
3      deletes = [k for k in state if k not in desired]
4      updates = [k for k in desired if k in state
5                  and desired[k]["properties"] != state[k]["saved"]]
6      return creates, updates, deletes
7
8  def apply(desired, state, creates, updates, deletes):
9      for key in deletes:
10         print(f" - {key}")
11         call("DELETE", url_for(state[key]["res"]))
12         del state[key]
13         save_state(state)          # save as we go, so partial failure
14     for key in creates + updates:  # leaves state matching reality
15         res = desired[key]
16         print(f" {'+' if key in creates else '~'} {key}")
17         call("PUT", url_for(res), res["properties"])
18         state[key] = {"res": res, "saved": res["properties"]}
19         save_state(state)
```

Three set operations, and a loop that acts on them. **That is the entire content of every progress bar you have ever watched.**

Step 3 – the vocabulary

```
src/3-state/engine.py
1 def url_for(res):
2     if res["type"] == "Microsoft.Resources/resourceGroups":
3         return f"{BASE}/subscriptions/{SUB}/resourcegroups/{res['name']}?api-version=2024-03-01"
4     if res["type"] == "Microsoft.Storage/storageAccounts/blobServices/containers":
5         return (f"{BASE}/subscriptions/{SUB}/resourceGroups/{res['resourceGroup']}"
6                 f"/providers/Microsoft.Storage/storageAccounts/{res['storageAccount']}"
7                 f"/blobServices/default/containers/{res['name']}?api-version=2023-05-01")
8     return (f"{BASE}/subscriptions/{SUB}/resourceGroups/{res['resourceGroup']}"
9            f"/providers/{res['type']}/{res['name']}?api-version=2023-05-01")
```

The script's URLs, folded into one function. This is the only type-specific code in the tool. Everything else is the loop.

The blob keeps its data-plane detour (a SAS the management plane signs), and the script's `sleep(20)` shrug lives on. All of `deploy.py` is in here now.

Step 3 – run it

```
zsh — byoiac

$ python src/3-state/engine.py up
+ create rg
+ create storage
+ create files
+ create hello
+ rg
+ storage
+ files
+ hello

$ python src/3-state/engine.py up
no changes.

$ cat state.json
{ "rg": { "res": { ... }, "saved": { "location": "eastus", "tags": { ... } } },
  "storage": { "res": { ... }, "saved": { ... } },
  "files": { "res": { ... }, "saved": { ... } },
  "hello": { "res": { ... }, "saved": { "contentType": "text/html", ... } } }

$ # edit infra.yaml: talk: kcdc → talk: kcdc2026
$ python src/3-state/engine.py plan
~ update rg
$ python src/3-state/engine.py up
~ update rg
~ rg
```

Run two sent **nothing**. `plan` called **zero** write APIs, and `state.json` is a **belief**, not a fact: nothing was checked.

The page is back

```
https://byoiacdemo2026.blob.core.windows.net/files/hello.html
```

The same page as minute five. This time it was deployed by a tool that **knows what it made**, and can run twice, update in place, and take it away again.

That was the promise at the top of the hour.

114

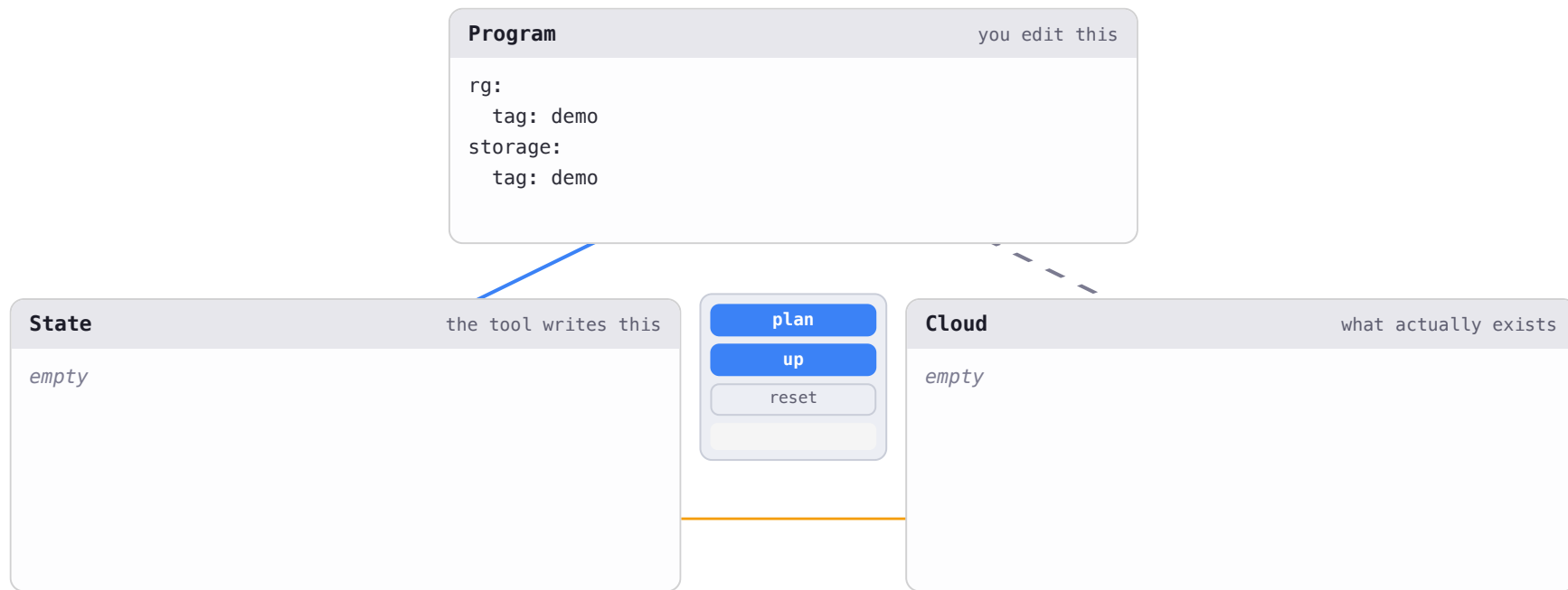
lines of Python

The script was fifty, and could deploy the page *once*. These 114 deploy the same page, and can run twice, and take things away.

State. Diff. Reconcile. **That is a working laC tool**, and now you know how all of them work.

The rest of the hour is the part nobody puts on a slide.

Now it plans deletes and reruns



Someone tidies the file

```
resources:
  rg:                # the group
    type: Microsoft.Resources/resourceGroups
    ...
  storage:          # the account
    type: Microsoft.Storage/storageAccounts
    ...
  files:            # the container
    type: ... /blobServices/containers
    ...
  hello:            # the page – the whole point
    type: blob
    ...
```

src/3-state/infra-tidied.yaml

Run it

```

zsh — byoiac

$ python src/3-state/engine.py destroy
- delete rg
- delete storage
- delete files
- delete hello
- hello
- files
- storage
- rg

$ cp src/3-state/infra-tidied.yaml src/3-state/infra.yaml
$ python src/3-state/engine.py up
+ create hello
+ create rg
+ create storage
+ create files

+ hello
Traceback (most recent call last):
...
File "src/3-state/engine.py", line 62, in blob_sas
    return out["serviceSasToken"]
TypeError: 'NoneType' object is not subscriptable

```

The tool created things in **file order**, and that won't scale.

A Go program couldn't have shipped this

Python – dicts remember insertion order.
Guaranteed, since 3.7.

```
for key in resources:    # file order
    create(key)          # works. every run.
```

The bug is **invisible**. You ship it.

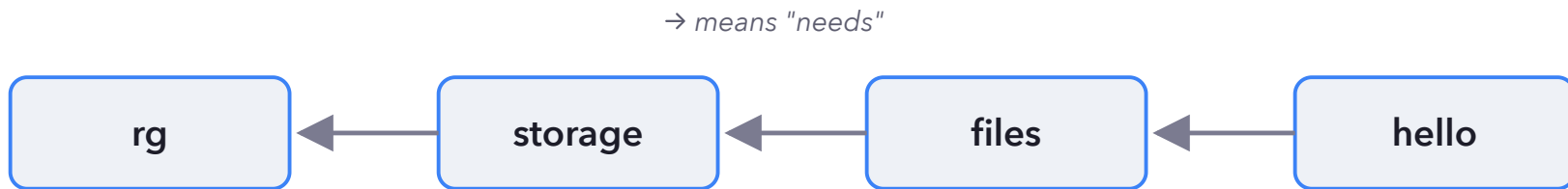
Go – map order is randomized. *On purpose*.

```
for name := range resources { // shuffled
    create(name)              // every run
}
```

Fails loudly, differently, **the first afternoon**.

Go's designers randomized maps precisely so nobody could build on this luck. Even so, the fix is **writing the order down**, not a stricter language.

The whole problem, four names



✗ `hello · rg · storage · files` – hello's parents don't exist yet. Crash.

✓ `rg · storage · files · hello` – every arrow satisfied.

The arrows are the **truth**. An order is just something you can *derive* from them.
So: store arrows. Derive order.

Step 4 – write the order down

src/3-state → src/4-graph – infra.yaml

```
1 resources:
2   rg:
3     name: byoiac-demo
4     ...
5   storage:
6     name: byoiacdemo2026
7     resourceGroup: byoiac-demo
8     ...
9   files:
10    name: files
11    storageAccount: byoiacdemo2026
12    ...
13   hello:
14     name: hello.html
15     container: files
16     ...
```

dependsOn: is the thing the script already had. It was the order of the lines.

Step 4 – and one of them isn't ARM at all

```
src/4-graph/infra.yaml
1  files:
2    type: Microsoft.Storage/storageAccounts/blobServices/containers
3    name: files
4    resourceGroup: byoiac-demo
5    storageAccount: byoiacdemo2026
6    dependsOn: [storage]
7    properties:
8      properties: {publicAccess: Blob}  # ARM nests container settings one level down
9
10  hello:
11    type: blob  # not an ARM type: files live on the data plane
12    name: hello.html
13    resourceGroup: byoiac-demo
14    storageAccount: byoiacdemo2026
15    container: files
16    dependsOn: [files]
17    properties:
18      contentType: text/html
19      content: |
20        <h1>Deployed by ~165 lines of Python</h1>
21        <p>KCDC 2026 - Build Your Own IaC</p>
```

The container is **still ARM**, just a deeper URL. The file is the **data plane**: a different API, different auth, a SAS the management plane signs for us. Still a PUT, though.

Step 4 – walking the graph

```
src/4-graph/engine.py

1  def ordered(resources): # depth-first topological sort
2      done, order = set(), []
3      def visit(key):
4          if key not in done:
5              done.add(key)
6              for dep in resources[key].get("dependsOn", []):
7                  visit(dep)
8              order.append(key)
9      for key in resources:
10         visit(key)
11     return order
12
13  def wait_ready(res): # 201/202 = "working on it" – poll until the cloud is done
14      while True:
15          code, actual = call("GET", url_for(res))
16          if code != 404 and (actual or {}).get("properties", {}).get(
17              "provisioningState") == "Succeeded":
18              return
19      time.sleep(2)
```

Parents first on the way **up**. Children first on the way **down**, same list, reversed.

Step 4 – run it

```
zsh — byoiac

$ rm -f state.json
$ python src/4-graph/engine.py up

+ create rg
+ create storage
+ create files
+ create hello

+ rg

+ storage
+ files
+ hello
```

~**21 seconds** between `+ rg` and `+ storage`. The storage account answered `202 Accepted` and we polled until it was real. The container and the file land in the second after that.

The page is back, and not by luck

<https://byoiacdemo2026.blob.core.windows.net/files/hello.html>

The same page is back, but this time the order was **derived from the arrows**, not inherited from the file. Reordering the file can't touch it.

Nothing can scramble it now.

Step 4 – the measured cost of "creating..."

	PUT returns	ready (provisioningState: Succeeded)
resource group	201 in 1.2s	1.4s
storage account	202 Accepted in 2.7s	20.8s
whole <code>up</code>		~21-23s

The API answered in under three seconds. The **resource** took twenty-one. Every progress bar you have ever watched is a poller.

Step 4 – and back down again

```
zsh — byoiac

$ python src/4-graph/engine.py destroy
- delete rg
- delete storage
- delete files
- delete hello

- hello
- files
- storage
- rg
```

Look at the order. The plan lists `rg` first. The **apply** deletes `hello` first: the file before its container, the container before its account. Reverse topological order, for free, from the same five lines.

You are here

- 1 **cli** – a script driving a tool
- 2 **api** – the same script, raw HTTP
- 3 **state** – store the state, derive the diff
- 4 **graph** – store the arrows, derive the order ◀ you are here
- 5 ?
- 6 ?
- ⋮
- N ?

What if something changes?



What we wrote, and what came back

infra.yaml — storage

```
storage:
  type: Microsoft.Storage/storageAccounts
  properties:
    location: eastus
    sku: {name: Standard_LRS}
    kind: StorageV2
    tags: {env: demo}
    properties:
      allowBlobPublicAccess: true
```

zsh — byoiac

```
$ curl -H "Authorization: Bearer $TOKEN" \
  "https://management.azure.com/.../storageAccounts/byoiacdemo2026"

{
  "sku": { "name": "Standard_LRS", "tier": "Standard" },
  "kind": "StorageV2",
  "location": "eastus",
  "tags": { "env": "demo" },

  "id": "/subscriptions/.../storageAccounts/byoiacdemo2026",
  "name": "byoiacdemo2026",
  "type": "Microsoft.Storage/storageAccounts",

  "properties": {
    "provisioningState": "Succeeded",
    "creationTime": "2026-08-13T16:05:05.2686429Z",
    "keyCreationTime": { "key1": "...", "key2": "... " },
    "primaryEndpoints": { "blob": "...", "dfs": "...", "file": "...",
                          "queue": "...", "table": "...", "web": "... " },

    "primaryLocation": "eastus",
    "statusOfPrimary": "available",
    "accessTier": "Hot",
    "minimumTlsVersion": "TLS1_0",
    "supportsHttpsTrafficOnly": true,
    "allowBlobPublicAccess": false,
    "allowCrossTenantReplication": false,
    "privateEndpointConnections": [],
    "networkAcls": { "defaultAction": "Allow", "bypass": "None", "ipRules": [] },
    "encryption": { "keySource": "Microsoft.Storage",
                    "services": { "blob": {}, "file": {} } }
  }
}
```

We sent 7. Azure answered with 48.

	fields we PUT	fields the GET returns
resource group	2 top-level (location , tags) · 4 keys total	6 top-level · 9 keys total
storage account	5 top-level (location , tags , sku , kind , properties) · 7 keys total	8 top-level · 48 keys total

A naive desired $\neq$ actual diff would propose "fixing" **42 fields nobody asked for**, several of which are read-only and will 400 if you try.

Step 5 – ask the cloud what is actually there

zsh — byoiac

```
$ python src/5-cliff/engine.py refresh  
  
= rg: cloud returned 6 top-level fields; we own 2  
= storage: cloud returned 8 top-level fields; we own 4  
= files: cloud returned 5 top-level fields; we own 1  
= hello: the data plane just answers with the bytes
```

`refresh` is one GET per resource. It is the moment the tool stops trusting its own memory, and the moment it discovers the cloud has opinions.

The fix, for exactly four resource types

```
src/5-cliff/engine.py

1 OWNED = { # per resource type: the properties that are OURS to manage
2     "Microsoft.Resources/resourceGroups": ["location", "tags"],
3     "Microsoft.Storage/storageAccounts": ["location", "tags", "sku", "kind"],
4     "Microsoft.Storage/storageAccounts/blobServices/containers": ["properties"],
5     "blob": ["contentType", "content"],
6 }
7
8 def owned(res): # our declared properties, minus anything not ours to manage
9     return {k: v for k, v in res["properties"].items() if k in OWNED[res["type"]]}
```

Six hand-written lines, for four resource types out of roughly **two thousand**.

The second verse

We sent this:

```
"sku": { "name": "Standard_LRS" }
```

Azure returned this:

```
"sku": { "name": "Standard_LRS", "tier": "Standard" }
```

`sku` is a field we **do** own, and it still comes back reshaped. An allowlist of *field names* isn't enough. You have to compare **shape-wise**.

src/5-cliff/engine.py

```
def project(shape, actual): # trim the cloud's answer to the shape we sent
    if isinstance(shape, dict) and isinstance(actual, dict):
        return {k: project(v, actual[k]) for k, v in shape.items() if k in actual}
    return actual           # (sku comes back with a tier we never sent)
```

Practical Problem

The loop was a hundred lines.
The schemas are the millions.

Which fields are mine · which are the server's · which are read-only · which come back reshaped

That knowledge is per-property, per-resource-type, across **~2,000 ARM types**, and then you do it all again for AWS.

Step 6 – drift, done right

```
src/6-drift/engine.py

1  def refresh(state):
2      for key in list(state):
3          res = state[key]["res"]
4          code, actual = call("GET", url_for(res))
5          if code == 404:                # deleted behind our back: forget it,
6              print(f" ! {key} vanished – deleted outside of us")
7              del state[key]            # and the next plan will offer to rebuild
8              continue
9          was, now = state[key]["saved"], project(owned(res), actual)
10         print(f" { '~' if was != now else '=' } {key}")
11         state[key]["saved"] = now
```

Let's see how we handle edits

zsh — not our tool

```
$ az group update -n byoiac-demo --set tags.env=HACKED -o none
```

One tag, changed behind our back. **Our state file has no idea.**

Step 6a – changed

```
zsh — byoiac

$ python src/6-drift/engine.py plan
no changes.

$ python src/6-drift/engine.py refresh
~ rg
= storage
= files
= hello

$ python src/6-drift/engine.py plan
~ update rg

$ python src/6-drift/engine.py up
~ update rg
~ rg
```

The first `plan` said **no changes**, because state is a *belief*, and nobody told it.

Let's see how we handle deletes

 zsh — not our tool

```
$ az group delete -n byoiac-demo --yes --no-wait
```

The whole stack. Gone. **Our state file has no idea.**

Step 6b – run it

```
zsh — byoiac

$ python src/6-drift/engine.py plan
no changes.

$ python src/6-drift/engine.py refresh
! rg vanished — deleted outside of us
! storage vanished — deleted outside of us
! files vanished — deleted outside of us
! hello vanished — deleted outside of us

$ python src/6-drift/engine.py plan
+ create rg
+ create storage
+ create files
+ create hello

$ python src/6-drift/engine.py up      # ~23s — rebuilt from nothing
```

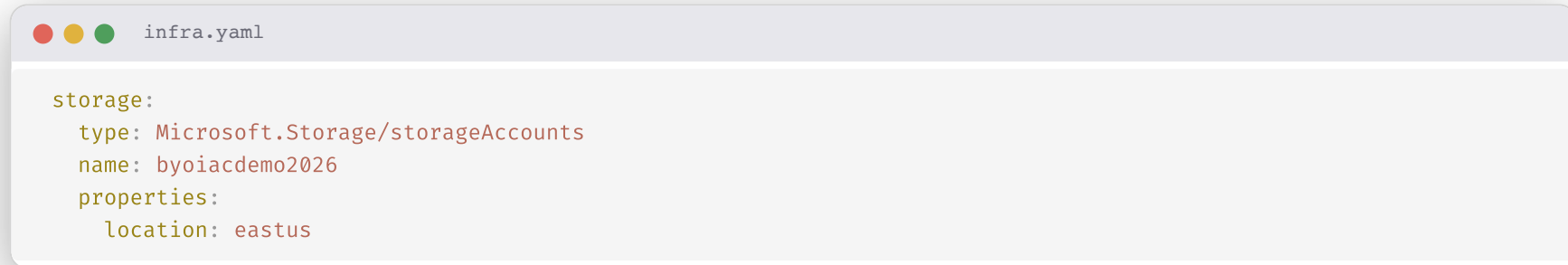
This beat is only possible **because of the cliff**. Without `OWNED`, every `refresh` would scream drift on 42 fields nobody touched, and the one real change would be buried.

You are here

- 1 **cli** – a script driving a tool
- 2 **api** – the same script, raw HTTP
- 3 **state** – store the state, derive the diff
- 4 **graph** – store the arrows, derive the order
- 5 **cliff** – know which fields are yours
- 6 **drift** – re-check the belief
- 7 ? ◀ **you are here**
- ⋮
- N ?

Next problem

Change one word in `infra.yaml` :



```
storage:
  type: Microsoft.Storage/storageAccounts
  name: byoiacdemo2026
  properties:
    location: eastus
```

What should `up` do?

You cannot move a storage account between regions. The "update" is secretly **delete + create**, and that one word just deleted an account with all its blobs in it.

Replace, in place, or swap?

One diff can be satisfied three different ways, and the tool has to pick:

update in place – PUT the change, the way our engine does. That works for a tag, but no PUT can move a region.

delete, then create – always works, and your storage account is *offline in between*.

create, then switch – build the new one, move traffic, delete the old. No downtime, but the name is taken, so the new one needs a *different name*.

...and it cascades – whatever depends on the replaced thing may have to rebuild too, in reverse dependency order.

Which move is legal, for which property, on which type: **per-property metadata again**. Real tools store it in the schema and plan the whole dance for you.

What about the other clouds?

Everything we built leaned on one mercy: **Azure is uniform**. One protocol, one auth, PUT everywhere.

AWS – three protocols before breakfast: per-service signing, different pagination, different error shapes, and IAM says "created" before it's true.

GCP – every create is a long-running operation to poll. Projects and quotas are first-class citizens.

Kubernetes – the API is *already* desired-state, so now you're reconciling against another reconciler.

All of them – their own retry rules, throttling, IDs, naming rules. None of it transfers.

Someone has to eat this list, per cloud, per service, per property, kept current forever. **That is what an IaC product actually is.**

We built 99% of the *idea* –

an afternoon well spent

how complete it **feels**

■ state · diff · reconcile · graph



how the **work** is actually distributed



▲ schemas × 2,000 types · replace rules · parallel applies · unknowns · retries · partial failure

– and 1% of the work.

So: use Pulumi

ours – infra.yaml

```
resources:
  rg:
    type: Microsoft.Resources/resourceGroups
    name: byoiac-demo
    properties:
      location: eastus
      tags: {env: demo, talk: kcdc}

  storage:
    type: Microsoft.Storage/storageAccounts
    name: byoiacdemo2026
    resourceGroup: byoiac-demo
    dependsOn: [rg]
    properties:
      location: eastus
      sku: {name: Standard_LRS}
      kind: StorageV2
      tags: {env: demo}
```

real Pulumi YAML

```
resources:
  rg:
    type: azure-native:resources:ResourceGroup
    properties:
      resourceGroupName: byoiac-demo
      location: eastus
      tags: {env: demo, talk: kcdc}

  storage:
    type: azure-native:storage:StorageAccount
    properties:
      accountName: byoiacdemo2026
      resourceGroupName: ${rg.name}

      location: eastus
      sku: {name: Standard_LRS}
      kind: StorageV2
      tags: {env: demo}
```

You **delete a line** when you graduate: `${rg.name}` is the edge, so `dependsOn` is gone.

Build your own *everything*

Your own database. Your own file system. Your own shell. Your own IaC.

The idea always fits in an afternoon. That's the fun part, and the learning part.

The cliff always comes. The moment it explodes in complexity is the payoff: now you know exactly what the real one is *for*, and you can read it fluently.

That's when you're allowed to stop. Don't run it in production. Keep the understanding.

Exercise for the reader

Program.cs

```
using Pulumi;
using Pulumi.AzureNative.Resources;
using Pulumi.AzureNative.Storage;
using Pulumi.AzureNative.Storage.Inputs;

return await Deployment.RunAsync(() =>
{
    var rg = new ResourceGroup("rg", new ResourceGroupArgs
    {
        ResourceGroupName = "byoiac-demo",
        Location = "eastus",
        Tags = { { "env", "demo" }, { "talk", "kcdc" } },
    });

    var storage = new StorageAccount("storage", new StorageAccountArgs
    {
        AccountName = "byoiacdemo2026",
        ResourceGroupName = rg.Name,
        Location = "eastus",
        Sku = new SkuArgs { Name = SkuName.Standard_LRS },
        Kind = Kind.StorageV2,
        Tags = { { "env", "demo" } },
    });
});
```

Thanks!



Adam Gordon Bell

Pulumi

✕ @adamgordonbell

☁ @adamgordonbell.bsky.social



The code + this talk
[github.com/adamgordonbell/
build-your-own-iac](https://github.com/adamgordonbell/build-your-own-iac)



My writing at Pulumi
[pulumi.com/blog/author/
adam-gordon-bell](https://pulumi.com/blog/author/adam-gordon-bell)